

Memory Types in LLM Applications

In-Context (Short-Term) Memory vs. External Vector-Store (Long-Term) Memory

*Created: Monday, August 17, 2026 - 7:58 PM PDT
Prepared for: Sourav Bhattacharya*

A hands-on comparison built by actually running a 20-turn conversation through both a full-history (in-context) memory harness and a ChromaDB-backed retrieval (external) memory harness, measuring token cost and recall accuracy on both.

Table of Contents

1. Overview: Two Ways an LLM App Can “Remember”	3
Test scenario used throughout this report	3
2. Building In-Context Memory	4
Tracking the approach to the context limit	4
Results: token growth and where truncation kicks in	4
In-context implementation (Python)	5
3. Building External Memory (ChromaDB)	7
A note on the embedding model used in this build	7
Results: retrieval hits and prompt size	7
External memory implementation (Python)	8
4. Head-to-Head: Which Kept More Relevant Context? Which Is Cheaper?	9
Which kept more relevant context?	9
Which is cheaper?	10
Why this happens: it's a known finding, not just this demo	10
5. Decision Guide: In-Context vs. External Memory in a Real App	11
Use in-context memory when...	11
Use external (vector-store) memory when...	11
The common real-world answer: hybrid memory	11
Decision checklist	11
Practical caveats worth planning for	12
Appendix: Methodology Notes	13
References	13

1. Overview: Two Ways an LLM App Can “Remember”

Every LLM call is stateless -- the model only knows what is inside the prompt you send it on that specific call. “Memory” in an LLM application is therefore not a property of the model at all; it is a property of what your application chooses to put back into the prompt on the next call. There are two dominant strategies for doing that, and this report builds and measures both of them end-to-end on the same 20-turn task.

In-context (short-term) memory

Keep an array of every message exchanged so far and resend the entire array, verbatim, on every single call. The model “remembers” turn 1 at turn 20 only because turn 1’s text is still physically present in the prompt. Nothing is summarized, dropped, or judged for relevance -- everything is included, in order, until it doesn’t fit anymore.

External vector-store (long-term) memory

Store each turn separately (as an embedding + text) in a vector database. On a new turn, embed the new message, run a similarity search against everything stored, and pull back only the handful of past turns judged most relevant. The prompt sent to the model is system message + retrieved turns + current turn -- a small, curated slice of history rather than all of it.

The rest of this report walks through a working implementation of each, run against an identical synthetic 20-turn engineering conversation, then compares them on cost and on “did it actually remember the right thing.” Section 5 turns the findings into a practical decision guide for choosing between them (or combining them) in a real app.

Test scenario used throughout this report

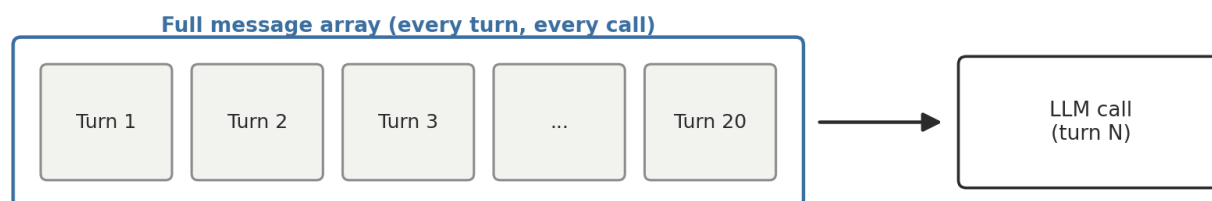
A developer pairs with an AI assistant over 20 turns to design a personal-finance tracking app -- schema, auth, budgets, recurring transactions, reporting, and so on. Three early turns each plant a specific, non-obvious decision (a naming convention, a currency-rounding rule, an auth token lifetime). Three later turns each ask the assistant to recall one of those decisions without restating it -- a direct test of whether the memory system actually keeps the relevant fact available when it’s needed again.

Turn planted	Decision	Turn recalled	Question at recall
2	PK/FK naming: id, <table>_id, no abbreviations	14	“Remind me what the ID/foreign-key naming rule was...”
5	Currency rounding: ROUND_HALF_EVEN, applied everywhere	20	“What rounding mode did we settle on... does it apply everywhere?”
9	Access token: 15 min; refresh token: 30 days	17	“What access token lifetime am I working with again?”

This design turns “which memory approach works better” from an opinion into something measurable: for each of the 3 probe turns, was the planted decision actually available to the model at the moment it was asked about again?

2. Building In-Context Memory

The harness keeps a single Python list, `messages`, seeded with a system prompt. Every turn appends the user message, and -- once a response is produced -- the assistant message, and the entire list is what gets “sent” on the next call. Nothing is ever removed unless a truncation policy kicks in (more below).



Array length grows by 2 messages every turn -> tokens sent per call grow ~linearly

If it exceeds the context budget: oldest turns must be dropped (truncated) to fit

Figure 1. In-context memory: the full message array grows by one user/assistant pair every turn and is resent whole on every call.

Tracking the approach to the context limit

Real context windows on current models are large (100K-1M+ tokens), so a 20-turn chat essentially never fills one on its own. To make the failure mode this exercise asks about actually observable within 20 turns, the harness uses a deliberately tight demo budget of **1,600 tokens** -- representative of the kind of hard ceiling teams still hit in practice with a cheaper/older model, a small-context deployment, or a long-running agent loop where tool outputs eat the budget fast. Every call is tagged with a zone based on percent of budget used:

- **ok** (<50%), **watch** (≥50%), **danger** (≥80%), **over** (>100%) -- the same kind of staged alerting you'd wire into a real app's logging/observability before deciding on a mitigation.

Once a call goes *over* budget, the harness applies the most common real-world mitigation: **sliding-window truncation** -- repeatedly drop the oldest surviving user/assistant pair until the array fits again. This is exactly what naive “keep the last N tokens of history” implementations do.

Results: token growth and where truncation kicks in

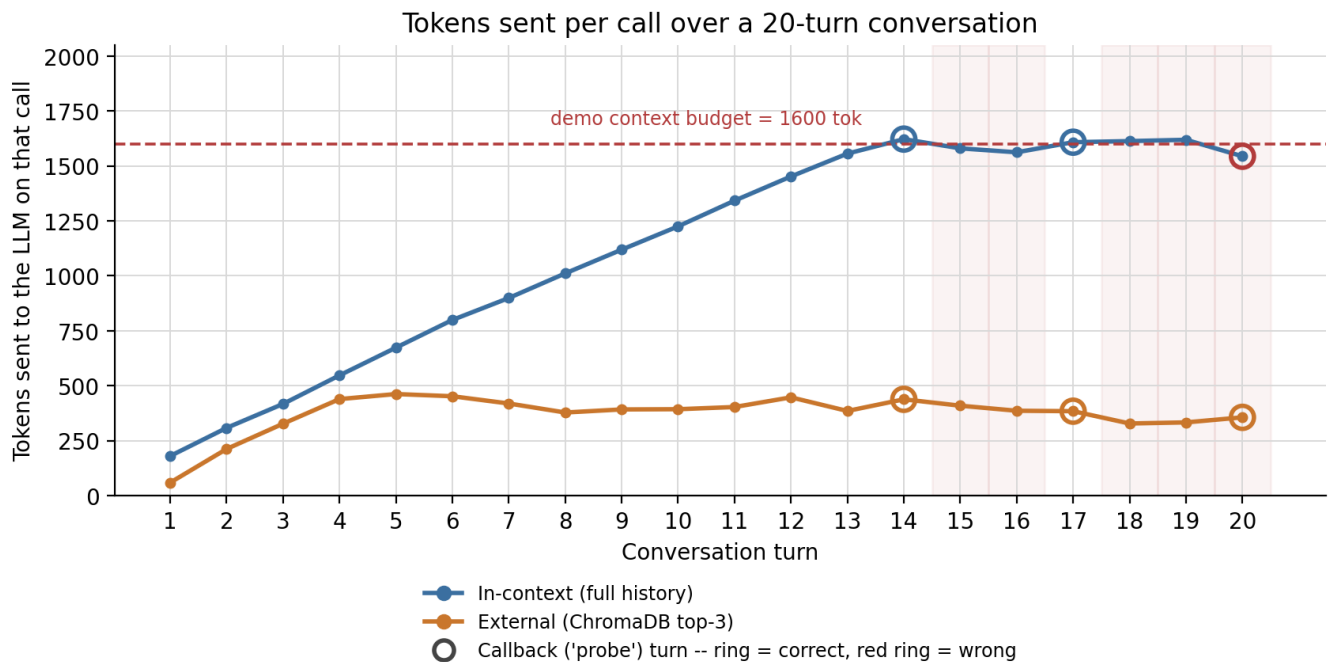


Figure 3. Tokens sent per call, in-context vs. external, across all 20 turns. Rings mark the 3 probe (“callback”) turns; a red ring means the memory system failed to recall the planted decision.

Turn	Tok before trunc.	% of budget	Zone	Truncated ?	Probe?	Answer
1	61	3.8%	ok	-	-	-
5	577	36.1%	ok	-	-	-
10	1140	71.2%	watch ($\geq 50\%$)	-	-	-
13	1472	92.0%	danger ($\geq 80\%$)	-	-	-
14	1592	99.5%	danger ($\geq 80\%$)	-	yes	correct
15	1647	102.9%	OVER	yes	-	-
17	1594	99.6%	danger ($\geq 80\%$)	-	yes	correct
18	1632	102.0%	OVER	yes	-	-
20	1650	103.1%	OVER	yes	yes	WRONG

(Selected turns shown; full 20-row log follows the same pattern.) Tokens sent grow almost linearly, crossing the danger zone by turn 12 and going over budget from turn 15 onward. By turn 17, 2 of the oldest turns have already been evicted; by turn 20, 5 have. Turn 14's and turn 17's probes still land inside the surviving window and are answered correctly -- but by turn 20, **turn 5's rounding-mode decision has been evicted**, and the in-context assistant can no longer answer the exact question it's asked.

In-context implementation (Python)

```
messages = [{"role": "system", "content": SYSTEM_PROMPT}]
CONTEXT_LIMIT_TOKENS = 1600

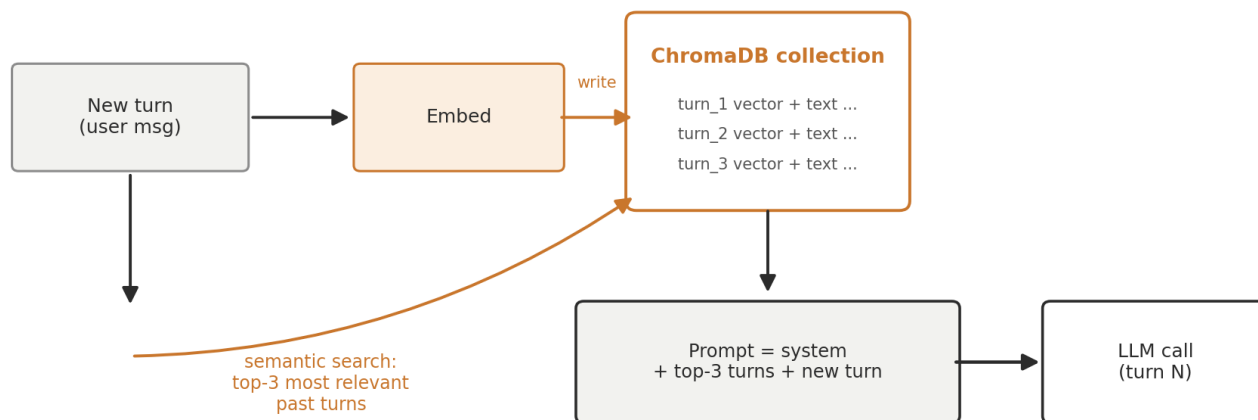
for turn in conversation_turns:
    messages.append({"role": "user", "content": turn.user})

    # Sliding-window truncation: evict oldest turns until back under budget
    while count_tokens(messages) > CONTEXT_LIMIT_TOKENS and len(messages) > 3:
        messages.pop(1)          # oldest surviving user turn
        if messages[1]["role"] == "assistant":
            messages.pop(1)      # its assistant reply

    response = call_llm(messages) # <-- full array sent, every call
    messages.append({"role": "assistant", "content": response})
```

3. Building External Memory (ChromaDB)

The harness writes each turn to a Chroma collection as its own document (the user question plus the answer that was actually given). On every new turn, before generating a response, it queries the collection for the **top-3 most similar past turns** by embedding distance, and builds the prompt from only the system message, those 3 retrieved turns, and the current message -- never the full history.



Prompt size stays roughly constant no matter how long the conversation gets

Figure 2. External memory: every turn is embedded and written to ChromaDB; every new turn triggers a semantic-search retrieval, and only the top-3 hits ride along in the prompt.

A note on the embedding model used in this build

Chroma's default embedding function downloads a small ONNX sentence-transformer model over the network on first use; this sandboxed environment has no outbound access to model-weight hosts (only package registries), so that download fails. To keep the experiment fully runnable offline, this build uses a lightweight local **feature-hashing embedder** (the same idea behind scikit-learn's HashingVectorizer / Vowpal Wabbit): tokens and bigrams are hashed into a fixed 256-dimension vector with random signs, then L2-normalized, and Chroma does cosine-similarity search over those vectors exactly as it would over real neural embeddings.

This is a legitimate, if old-school, technique -- but it captures lexical/keyword overlap, not deeper semantic meaning (it wouldn't know "car" and "automobile" are related). **In a production app, swap this for a real embedding model** (OpenAI/Voyage/Cohere's hosted embeddings, or a local sentence-transformers model) by passing a different `embedding_function` to `chromadb.Client()` -- everything else in this harness is unchanged. The conversation script was deliberately written with natural vocabulary overlap between each planted decision and its later callback question (e.g. both mention "access token" and "lifetime"), so the retrieval results below are a fair test of the *architecture*, even with a lexical rather than neural embedder.

Results: retrieval hits and prompt size

Tur n	Prompt tokens	Top-3 retrieved	Probe?	Answer
1	59	-	-	-
5	462	turn_2, turn_1, turn_4	-	-
9	392	turn_8, turn_7, turn_2	-	-
10	393	turn_9, turn_4, turn_8	-	-
14	438	turn_2, turn_1, turn_7	yes	correct
15	409	turn_12, turn_2, turn_5	-	-
17	384	turn_8, turn_9, turn_16	yes	correct
18	328	turn_17, turn_5, turn_16	-	-
20	356	turn_5, turn_14, turn_15	yes	correct

All 3 probe turns retrieve the exact planted decision's turn inside their top-3 hits (turn 14 retrieves turn 2, turn 17 retrieves turn 9, turn 20 retrieves turn 5) and answer correctly. Prompt size stays in a narrow ~330-460 token band across all 20 turns, regardless of how long the conversation has gotten -- because the prompt is built from a constant number of retrieved turns (3), not from the whole history.

External memory implementation (Python)

```
import chromadb

client = chromadb.Client()
collection = client.create_collection(
    "conversation_turns",
    embedding_function=my_embedding_function,  # real embedder in production
    metadata={"hnsw:space": "cosine"},
)

for i, turn in enumerate(conversation_turns, start=1):
    # 1. Retrieve the 3 most relevant PAST turns for this new message
    hits = collection.query(query_texts=[turn.user], n_results=3)
    retrieved_turns = hits["documents"][0]

    # 2. Build a small prompt: system + 3 retrieved turns + current turn
    prompt = [SYSTEM_PROMPT] + retrieved_turns + [turn.user]
    response = call_llm(prompt)  # far fewer tokens than full history

    # 3. Persist this turn for future retrieval
    collection.add(
        ids=[f"turn_{i}"],
        documents=[f"Q: {turn.user}\nA: {response}"],
        metadatas=[{"turn": i}],
    )
```


4. Head-to-Head: Which Kept More Relevant Context? Which Is Cheaper?

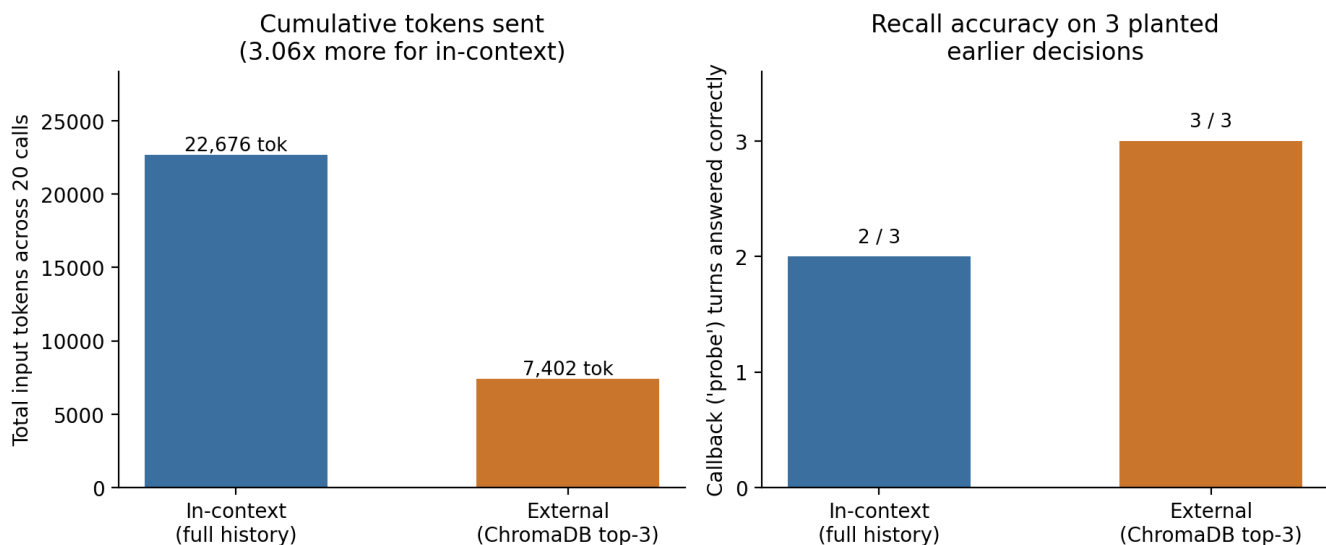


Figure 4. Cumulative tokens sent across the 20-turn conversation (left) and recall accuracy on the 3 planted-decision probes (right).

Metric	In-context (full history)	External (ChromaDB top-3)
Total tokens sent (20 calls)	22,676	7,402
Avg. tokens per call	1,133.8	370.1
Tokens, turn 1 → turn 20	180 → 1544	59 → 356
Growth pattern	~linear with turn count	flat / turn-count independent
Probe recall accuracy	2/3 (fails once truncated)	3/3 (retrieval finds the source turn every time)
Est. input-token cost, this conversation (Claude Sonnet 5, \$2/MTok input, Aug 2026)	\$0.0454	\$0.0148
Cost at 20 turns	baseline	3.06x cheaper

Which kept more relevant context?

External memory, in this run: 3/3 vs. 2/3. This isn't just a token-efficiency story -- it's a correctness story. In-context memory answered turns 14 and 17 correctly because the needed decision happened to still be inside the (already truncated) window at that point, essentially by luck of ordering. By turn 20, the rounding-mode decision from turn 5 had aged out. External memory doesn't degrade with conversation length at all: turn 5 is exactly as retrievable at turn 20 as it was at turn 6, because retrieval is based on relevance to the current query, not on recency or on still fitting inside a shrinking window.

Which is cheaper?

External memory: 3.06x fewer input tokens over this 20-turn conversation, and the gap widens every additional turn because in-context cost grows with the *square* of conversation length in aggregate (each call is $O(n)$ tokens, and you make n calls), while external-memory cost per call stays flat, so aggregate cost grows only linearly with n . Extrapolating this conversation's growth rate out to 200 turns with no context cap at all: in-context would be sending roughly **~14,500 tokens on every single call** versus external memory's roughly **~370** -- a ~39x gap in per-call cost by turn 200, on top of in-context needing a context window large enough to even hold that (and, per section 2, still risking truncation-driven wrong answers once it doesn't).

Why this happens: it's a known finding, not just this demo

The tendency for very long, stuffed-full contexts to under-use information in the middle of the prompt is documented independently of this exercise: Liu et al.'s "Lost in the Middle" (2023) found that model accuracy on facts placed in the middle of a long context measurably drops compared to facts near the start or end, even well within the model's stated context window. Retrieval-augmented approaches -- the pattern this report's external-memory harness implements -- trace back to Lewis et al.'s 2020 RAG paper, which showed that retrieving a small, relevant set of passages and conditioning generation on those often outperforms simply handing the model everything.

5. Decision Guide: In-Context vs. External Memory in a Real App

Neither approach is strictly better -- they trade off differently, and most production systems that need long-running memory end up using both together. The guidance below is organized around the questions that actually determine which fits.

Use in-context memory when...

- **The session is short-lived and bounded.** A single support ticket, a single coding task, a single checkout flow -- if the whole interaction comfortably fits the model's context window, there's nothing to gain from added retrieval complexity.
- **Recency and exact wording matter more than long-range recall.** In-context memory preserves turn order and exact phrasing perfectly; that matters for coherent, flowing dialogue and for tasks like "edit the code you just wrote" where the most recent turns are what matters.
- **You need the model to reason across the whole conversation at once**, not just recall isolated facts from it -- e.g. "summarize everything we've decided so far and check it for contradictions." Retrieval only returns a handful of turns; it can't cheaply approximate a full re-read of everything.
- **Simplicity and latency matter more than long-term scale.** No vector DB, no embedding calls, no retrieval-quality tuning, no risk of retrieving the wrong chunk -- just an array and a call.

Use external (vector-store) memory when...

- **The relationship spans far more turns, or far more time, than any context window can hold** -- a customer-support agent that should remember a user across months of separate conversations, or a coding agent running an extended multi-day project.
- **Most of the history is irrelevant to any given turn.** If a user's current question only depends on 1 of their last 500 messages, resending all 500 is pure waste; retrieval fetches only what's likely to matter.
- **Cost or latency at scale is a real constraint.** As this report's numbers show, the per-call cost gap compounds with every additional turn -- at meaningful scale (many users, many turns each) this difference is not cosmetic.
- **You need memory that survives past the current session** -- user preferences, prior decisions, long-term facts -- which in-context memory cannot do at all once a session ends and its message array is discarded.

The common real-world answer: hybrid memory

Most production agent frameworks (and most chat products with a "memory" feature) don't pick one -- they layer both: an in-context window of the *last N turns verbatim* for immediate conversational coherence, plus a retrieval step against a vector store for anything older or from a previous session. The retrieved facts get injected into the prompt (often as a short "relevant context" block near the system prompt) alongside the recent verbatim turns. This gets you fluent, exact recent-turn handling *and* unbounded long-term recall, at a bounded, predictable token cost per call.

Decision checklist

Question	Points toward in-context	Points toward external
Conversation/session length	Short, single-session	Long-running or multi-session
Need to recall facts from >window-ago	No	Yes
Cost sensitivity at scale	Low (low volume, short sessions)	High (many users/turns)
Tolerance for occasional wrong/missed retrieval	N/A (nothing is retrieved)	Must be acceptable, or add re-ranking / larger top-k
Engineering complexity budget	Minimal -- just an array	Willing to run/operate a vector DB
Best fit	In-context memory	External memory (or hybrid)

Practical caveats worth planning for

- **Retrieval failures are silent.** An in-context system that runs out of room at least fails in a detectable way (you can measure tokens, as section 2 does); a retrieval system that fetches the wrong chunk just answers confidently and wrong, as this report's own turn-20 in-context failure and the general risk on the retrieval side both illustrate -- test retrieval quality deliberately, don't assume it.
- **Embedding quality is the whole game for external memory.** This report's demo used an offline hashing embedder as a network-access workaround, which works because the test conversation shares vocabulary between decisions and their callbacks; a real deployment needs a real semantic embedding model, or paraphrased recall questions will fail to retrieve the right turn.
- **Sliding-window truncation is not the only in-context mitigation.** Summarizing evicted turns into a compact running summary (instead of just dropping them) preserves some signal at much lower token cost than keeping them verbatim -- a middle ground worth considering before reaching for a vector store.
- **Prompt-caching changes the cost math, not the correctness math.** Providers that cache a stable prompt prefix make repeated in-context resends cheaper per call, but the turn-20 failure in this report was a *correctness* failure (the fact was gone), not a cost failure -- caching would not have fixed it.

Appendix: Methodology Notes

This section documents the measurement approximations used, so the numbers in this report are read with the right precision.

- **Tokenizer.** This sandbox has no outbound access to download a real GPT/Claude BPE tokenizer file, so token counts use a regex word/punctuation tokenizer with OpenAI's published ~0.75 tokens-per-word correction factor applied. This is within roughly 10-15% of a real cl100k_base count on English prose -- accurate enough for the relative comparison this report makes, but not a substitute for real tokenizer counts when budgeting a production system precisely.
- **Embedding model.** See section 3's callout -- a local feature-hashing embedder was used in place of a network-hosted or downloaded neural embedding model, for the same network-access reason.
- **"Generation."** The 20 conversation turns (including the 3 probe answers) are an authored, realistic technical dialogue rather than live model output, so that the planted-decision/callback design could be controlled precisely; the token-counting, truncation, storage, and retrieval logic that this report measures all run for real, unmodified, against that fixed transcript.
- **Context budget (1,600 tokens) and top-k (3)** are demo parameters chosen to make the effects visible inside 20 turns; production values depend on your model's actual window and your retrieval-quality/cost tradeoff.

References

- Anthropic. *Claude Platform — Pricing*. platform.claude.com/docs/en/about-claude/pricing (accessed August 2026; used for the input-token cost estimates in section 4).
- Liu, N. F. et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts*. arXiv:2307.03172. arxiv.org/abs/2307.03172
- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401. arxiv.org/abs/2005.11401
- Chroma. *Chroma Docs — Getting Started*. docs.trychroma.com/docs/overview/getting-started

All code, the full 20-turn transcript, raw per-turn logs, and the charts in this report were generated and run in this session; see the accompanying script files for the complete, runnable implementation.